AIuthor

Antigravity Implementation Plan

Backend-first module-wise build guide for React + FastAPI + PostgreSQL + pgvector + LangGraph

Document Area	Purpose
Backend First	Define project architecture, database, APIs, agent orchestration, RAG, memory, observability, export and evals before UI polish.
React UI Second	Build a practical UI for creating runs, monitoring agents, viewing memory/traces/evals, and downloading outputs.
Antigravity Usage	Use the included prompts module-by-module. Do not ask Antigravity to generate the whole system in one pass.
Testing Included	Every module includes unit, integration, and acceptance tests so the MVP remains stable.

Important build principle: **Do not build a fancy UI around a single long prompt. Build a real agentic workflow with structured memory, trace logging, prompt logs, and repair flow.**

Table of Contents

1. Project Purpose and Success Criteria

AIuthor is an agentic long-form book generation system. The system takes a user brief such as topic, reader profile, length, tonality, and genre, then produces a publication-ready book with complete front matter, chapters, back matter, consistent voice, memory-backed continuity, fact grounding, DOCX/PDF export, and traceable execution logs.

The main goal is not to prove that an LLM can write text. The goal is to prove that the engineering system can orchestrate multiple agents, manage memory outside the context window, ground factual claims, evaluate outputs, and repair dependent content when the book structure changes.

Success Area	What Must Work
Agentic design	Planner, Researcher, Writer, Humanizer, Editor, Fact Checker, Memory Keeper and Assembler must run as separate coordinated units.
Memory	Fact registry, concept bible, character bible, callback index, tone fingerprint and decision log must be persistent.
Grounding	Non-fiction chapters must use retrieved evidence and avoid unsupported factual claims.
Book quality	The output must look like a real book with front matter, body chapters and back matter.
Repairability	Inserting a new chapter must repair TOC, callbacks and glossary.
Observability	Every run must create agent trace, prompt logs, memory I/O logs and token/cost ledger.

2. Target Architecture

The project should use React for the frontend, FastAPI for the backend, PostgreSQL for transactional data, pgvector for semantic search, and LangGraph for multi-agent orchestration.

```
React UI
  -> FastAPI API Layer
      -> LangGraph Orchestrator
          -> Planner Agent
          -> Researcher Agent -> pgvector Retrieval
          -> Writer Agent
          -> Humanizer Agent
          -> Editor Agent
          -> Fact Checker Agent
          -> Memory Keeper Agent -> PostgreSQL Memory Tables
          -> Assembler Agent -> DOCX/PDF
          -> Evals Runner
      -> Trace Logs / Prompt Logs / Memory I/O / Token Cost Ledger
```

Layer	Technology	Reason
Frontend	React + Vite + Tailwind	Fast to build, clean workflow UI, easy state management.
Backend	FastAPI	Simple async APIs, Pydantic models, good for AI services.
Database	PostgreSQL	Reliable transactional storage for books, runs, chapters, memory and logs.

Vector DB	pgvector	Keeps vectors close to core data and is enough for MVP.
Orchestration	LangGraph	Supports stateful graph execution, loops and failure recovery.
Export	python-docx + LibreOffice	Good for DOCX and PDF delivery.

3. Implementation Strategy for Antigravity

Use Antigravity in small controlled modules. Do not ask it to build the whole app at once. Each prompt should tell it exactly which module to implement, which files to create, what tests to add, and when to stop.

- Start with backend foundation before UI.
- Build database models before agents.
- Build prompt registry before agent logic.
- Build tracing before running full workflows.
- Build one agent at a time and test output schemas.
- Build Test D repair logic before final UI polish.
- After every module, run tests and show changed files.

Antigravity Rule	Why
One module per prompt	Prevents uncontrolled architecture drift.
Always ask for tests	Keeps implementation verifiable.
Ask for file-by-file output	Makes review easier.
Ask not to remove existing code	Prevents regression.
Ask for stop point	Makes module completion clear.

4. Backend Module 0 - Repository and Environment Setup

Create the base repository structure, Docker environment, dependencies and project layout.

0.1 Repository structure

- Create backend/, frontend/, docs/, prompts/, sample_books/, runs/ folders.
- Add README.md with setup commands.
- Add .env.example for API keys and database URLs.

0.2 Docker setup

- Create docker-compose.yml with postgres, redis, backend and frontend services.
- Enable PostgreSQL extension pgvector in init script.
- Expose backend on 8000 and frontend on 5173.

0.3 Python dependencies

- Add fastapi, uvicorn, sqlalchemy, alembic, psycopg, pydantic, python-dotenv, langgraph, openai or provider SDK, python-docx, pytest.
- Create requirements.txt or pyproject.toml.

0.4 Frontend placeholder

- Create React placeholder folder but do not implement UI yet.
- Backend remains first priority.

Testing for this module

- docker compose config should pass.
- Backend app should start with /health endpoint.
- PostgreSQL container should accept connections.
- pgvector extension should be available.

Module deliverables

- Clean repository skeleton.
- docker-compose.yml.
- .env.example.
- README setup section.

5. Backend Module 1 - FastAPI Core and Configuration

Create the backend app foundation with health checks, app configuration, database session and route grouping.

1.1 App entrypoint

- Create app/main.py.
- Register CORS middleware for React.
- Add /health and /api/version endpoints.

1.2 Settings

- Create app/config.py using pydantic settings.
- Read DATABASE_URL, REDIS_URL, OPENAI_API_KEY, MODEL_GENERATION, MODEL_EVALUATION.

1.3 Database session

- Create app/database.py.
- Implement SQLAlchemy engine and session dependency.
- Add get_db dependency for routes.

1.4 Error handling

- Add global exception handler.
- Return clean JSON errors with trace_id.

Testing for this module

- GET /health returns ok.
- Missing env var is handled clearly.
- Database dependency can open and close a session.
- CORS config includes frontend origin.

Module deliverables

- FastAPI app runs.
- Health API works.
- Basic test suite works.

6. Backend Module 2 - PostgreSQL and pgvector Database Layer

Define all core tables required for books, runs, chapters, sections, memory, RAG chunks, traces and evals.

2.1 Alembic setup

- Initialize Alembic migrations.
- Configure metadata import from SQLAlchemy models.
- Create initial migration.

2.2 Core book tables

- book_projects, book_runs, chapters, book_sections, export_files.

2.3 Memory tables

- fact_registry, concept_bible, character_bible, callback_index, tone_fingerprints, decision_log.

2.4 RAG tables

- source_documents, document_chunks with vector(1536), retrieval_logs.

2.5 Observability tables

- agent_traces, prompt_logs, memory_io_logs, token_cost_ledger, eval_results.

Testing for this module

- Migration creates all tables.
- pgvector extension exists.
- Can insert one book and one chapter.
- Can insert one embedding row.
- Foreign keys reject invalid records.

Module deliverables

- SQLAlchemy models.
- Alembic migration.
- DB smoke tests.

7. Backend Module 3 - Pydantic Schemas and API Contracts

Create strict request and response schemas for API and agents so data does not become loose text blobs.

3.1 Book schemas

- BookCreateRequest, BookResponse, ChapterResponse, BookSectionResponse.

3.2 Run schemas

- RunStartRequest, RunStatusResponse, AgentStatusItem.

3.3 Agent schemas

- BookBrief, BookOutline, ChapterContract, EvidencePack, FactCheckResult, MemoryUpdate.

3.4 Eval schemas

- EvalResultResponse, StructureEvalDetails, ToneEvalDetails, FactEvalDetails.

3.5 Export schemas

- ExportFileResponse, ExportStatusResponse.

Testing for this module

- Invalid tone returns validation error.
- Chapter count must be positive.
- EvidencePack requires source_chunk_id for grounded facts.
- FactCheckResult must use allowed statuses only.

Module deliverables

- app/schemas/*.py.
- Schema unit tests.
- API contract examples.

8. Backend Module 4 - Prompt Registry and Prompt Dossier System

Create a prompt system where every agent prompt is stored as a real file and logged during execution.

4.1 Prompt folder

- Create prompts/planner.md, researcher.md, writer.md, humanizer.md, editor.md, fact_checker.md, memory_keeper.md, assembler.md, evaluator.md.

4.2 Prompt metadata

- Each prompt must include purpose, inputs, output schema, rules, failure modes and why this prompt exists.

4.3 Prompt loader

- Create app/prompts/registry.py.
- Load prompt text by agent name.
- Support variable injection safely.

4.4 Prompt dossier export

- Create a backend service to assemble all prompts into a standalone markdown/PDF dossier later.

Testing for this module

- Every required agent has a prompt file.
- Prompt loader fails clearly for missing prompt.

- Humanizer prompt contains banned phrase rules.
- Fact-checker prompt contains citation-or-soften rule.

Module deliverables

- Prompt files.
- Prompt loader service.
- Prompt validation tests.

9. Backend Module 5 - Observability: Trace, Prompt, Memory I/O, Token Cost Ledger

Implement logging before full agent execution so every future module is traceable.

5.1 Trace service

- Create trace record at agent start.
- Update trace record at completion or failure.
- Store input_summary and output_summary.

5.2 Prompt log service

- Store full prompt_text, model_name, input_payload and output_payload.

5.3 Memory I/O service

- Log every memory read/write operation with memory_type and payload.

5.4 Token/cost ledger

- Store input_tokens, output_tokens, model name and estimated cost.
- Make model pricing configurable.

Testing for this module

- Agent trace created on fake agent run.
- Prompt log stores exact prompt text.
- Memory read and write create logs.
- Failed agent stores error.

Module deliverables

- app/services/trace_service.py.
- app/services/ledger_service.py.
- routes for trace viewing.

10. Backend Module 6 - RAG and Research Source Pipeline

Implement grounded retrieval for non-fiction chapters using PostgreSQL + pgvector.

6.1 Source ingestion

- Create source_documents records.

- Chunk text into 500-900 token chunks.
- Store chunk metadata.

6.2 Embeddings

- Create embedding service.
- Store embedding vector in document_chunks.
- Use 1536 dimensions if using text-embedding-3-small.

6.3 Retrieval

- Implement cosine similarity retrieval.
- Add keyword search fallback.
- Return top_k chunks with scores.

6.4 Evidence pack builder

- Convert retrieved chunks into EvidencePack schema.
- Mark missing evidence instead of fabricating.

6.5 Retrieval logging

- Log query, top_k, returned chunk IDs and scores.

Testing for this module

- Ingest sample source into chunks.
- Embeddings are saved.
- Similarity search returns relevant chunk.
- Missing evidence is reported.
- Researcher cannot invent source IDs.

Module deliverables

- rag/chunker.py.
- rag/embeddings.py.
- rag/retriever.py.
- EvidencePack service.

11. Backend Module 7 - Memory System

Build persistent memory outside the model context window for continuity across chapters.

7.1 Fact registry

- Store claim, source_chunk_id, confidence, status and used chapter.

7.2 Concept bible

- Store concepts, definitions, first_chapter and related terms.

7.3 Character bible

- Store named characters, traits, relationships, arc summary and chapter appearances.

7.4 Callback index

- Store source_chapter, target_chapter, concept and callback text.

7.5 Tone fingerprint

- Store tone rules, rhythm, lexical patterns, banned phrases and examples.

7.6 Decision log

- Store major design/runtime decisions and reasons.

7.7 Memory reader

- Retrieve relevant facts, concepts, callbacks, characters and tone for current chapter.

Testing for this module

- After chapter generation, facts are saved.
- Concept glossary terms are extracted.
- Character appears consistently across chapters.
- Callback index returns relevant prior concept.
- Memory retrieval does not return unrelated records.

Module deliverables

- memory/store.py.
- memory/services.py.
- memory extraction prompts.
- Memory API endpoints.

12. Backend Module 8 - LangGraph Orchestration

Create the real multi-agent workflow with state, routing, retries and repair loops.

8.1 State object

- Create BookState with book_id, run_id, brief, outline, current_chapter, evidence_pack, drafts, fact_check_result and errors.

8.2 Graph nodes

- planner_node, researcher_node, writer_node, humanizer_node, editor_node, fact_checker_node, memory_keeper_node, assembler_node, evals_node.

8.3 Conditional edges

- If fact-check fails, go back to researcher or writer repair.
- If all chapters done, go to assembler.
- If fatal error, mark run failed.

8.4 Run service

- Start workflow from API.
- Update current_agent and status in book_runs.
- Support polling from UI.

8.5 Event streaming

- Optional: Server-Sent Events or WebSocket for run monitor.

Testing for this module

- Graph can run a mocked workflow.
- Fact-check failure routes to repair path.
- Run status updates per agent.
- Fatal error marks run failed.
- All chapters route to assembler.

Module deliverables

- graph/state.py.
- graph/workflow.py.
- services/run_service.py.

13. Backend Module 9 - Agent Implementation

Implement each required agent with strict input/output contracts and full logging.

9.1 Planner Agent

- Create book title, subtitle, outline, chapter contracts, front matter plan and back matter plan.

9.2 Researcher Agent

- Read chapter contract, generate search queries, retrieve evidence from pgvector and output EvidencePack.

9.3 Writer Agent

- Write chapter draft using outline, evidence and memory. Avoid unsupported claims.

9.4 Humanizer Agent

- Remove AI tells, improve rhythm, add reader connection and preserve facts.

9.5 Editor Agent

- Improve structure, grammar, headings, repetition and transitions.

9.6 Fact Checker Agent

- Extract claims, compare with evidence, mark supported/partial/unsupported and trigger repair.

9.7 Memory Keeper Agent

- Extract facts, concepts, characters, callbacks and update memory tables.

9.8 Assembler Agent

- Build complete book sections and pass data to exporter.

Testing for this module

- Each agent returns valid schema.

- Humanizer removes banned phrases.
- Fact checker flags unsupported claims.
- Memory keeper writes expected tables.
- Assembler checks required sections.

Module deliverables

- agents/*.py.
- prompt files.
- agent unit tests.

14. Backend Module 10 - Chapter Insertion and Self-Healing Repair

Implement Test D: insert a new chapter and automatically repair dependent structures.

10.1 Insert API

- POST /api/books/{book_id}/chapters/insert.
- Input after_chapter, title and purpose.

10.2 Outline repair

- Insert new chapter contract.
- Renumber all later chapters.
- Update chapter ordering in DB.

10.3 New chapter generation

- Run Researcher -> Writer -> Humanizer -> Editor -> Fact Checker -> Memory Keeper only for new chapter.

10.4 Callback repair

- Update callback target_chapter numbers.
- Detect callbacks that now point to wrong chapters.
- Regenerate affected callback text if needed.

10.5 Glossary repair

- Add new concepts from inserted chapter.
- Remove unused terms if necessary.
- Rebuild glossary section.

10.6 TOC and export repair

- Rebuild TOC.
- Re-export DOCX/PDF.
- Run insertion eval.

Testing for this module

- New chapter appears after Chapter 4.
- Old Chapter 5 becomes Chapter 6.
- TOC order is correct.

- Callbacks point to correct chapter numbers.
- Glossary includes new terms.
- Exports regenerate successfully.

Module deliverables

- chapter_insert_service.py.
- repair_service.py.
- Test D runner.

15. Backend Module 11 - DOCX and PDF Export

Generate publication-style DOCX and PDF outputs with required book sections.

11.1 Book assembly model

- Collect front matter, chapters and back matter in correct order.

11.2 DOCX generation

- Use python-docx.
- Set heading styles.
- Add page breaks.
- Add TOC placeholder.
- Add front matter and back matter.

11.3 Page numbering approach

- Use section breaks for front matter and body.
- Roman numeral front matter if time permits.
- Normal page numbers from introduction/body.

11.4 PDF conversion

- Convert DOCX to PDF using LibreOffice headless.
- Store export file metadata.

11.5 Export API

- GET /api/exports/{book_id}/docx.
- GET /api/exports/{book_id}/pdf.

Testing for this module

- DOCX exists.
- PDF exists.
- Book has front matter, chapters and back matter.
- Glossary generated from concept bible.
- No fake references in references section.

Module deliverables

- exporters/docx_exporter.py.
- exporters/pdf_exporter.py.

- Export APIs.

16. Backend Module 12 - Evals and Quality Gates

Create automated evaluation so the system can prove quality instead of relying on manual opinion.

12.1 Structure eval

- Check all required sections exist.
- Check chapter count.
- Check back matter exists.

12.2 Tone eval

- Use rule-based checks plus optional LLM judge.
- Compare against tone fingerprint.

12.3 AI tell eval

- Detect banned phrases and mechanical patterns.

12.4 Fact eval

- Compare final claims to fact registry and evidence pack.
- Count unsupported claims.

12.5 Callback eval

- Check callback index and ensure callbacks appear in target chapters.

12.6 Insertion eval

- Check Test D TOC, callbacks and glossary self-heal.

Testing for this module

- Missing section fails structure eval.
- Banned phrase triggers AI tell fail.
- Unsupported claim triggers fact eval fail.
- Inserted chapter passes/fails based on actual repaired state.

Module deliverables

- evals/*.py.
- eval_results table records.
- Eval report API.

17. Backend Module 13 - Test Case Runners A-D

Create repeatable scripts/API commands to generate all mandatory assessment test cases.

13.1 Test A runner

- 10-chapter beginner personal finance guide, conversational tone.

13.2 Test B runner

- 5-chapter novella, storyteller tone, two named characters.

13.3 Test C runner

- Regenerate Test A Chapter 3 in Academic, Motivational and Witty tones.

13.4 Test D runner

- Insert new chapter between Test A Chapter 4 and 5 and repair outputs.

13.5 Trace bundle exporter

- Export trace.json, prompt_log.json, memory_io_log.json, token_cost_ledger.csv, eval_report.json.

Testing for this module

- Each runner creates a completed run.
- Each runner creates DOCX and PDF.
- Trace bundle exists for every run.
- Evals exist for every run.

Module deliverables

- scripts/run_test_a.py.
- scripts/run_test_b.py.
- scripts/run_test_c.py.
- scripts/run_test_d.py.

18. Frontend Module 1 - React Project Setup

Create the Vite React app, routing, API client, shared layout and design tokens.

1.1 Vite setup

- Create frontend with React + Vite.
- Install react-router-dom, axios or fetch wrapper, Tailwind CSS.

1.2 Routing

- Routes: /, /new-book, /runs/:runId, /runs/:runId/book, /runs/:runId/memory, /runs/:runId/traces, /runs/:runId/evals, /runs/:runId/exports.

1.3 API client

- Create src/api/client.js.
- Centralize backend base URL.
- Handle loading and error states.

1.4 Layout

- Sidebar navigation.
- Top header with project name and current run status.

Frontend tests

- React app starts.
- Routes render placeholder pages.
- API client handles backend error.
- Layout responsive basic check.

19. Frontend Module 2 - New Book Brief UI

Create the form to launch book generation.

2.1 Brief form fields

- Topic, reader profile, genre, tone, target chapters, words per chapter, special instructions.

2.2 Tone selector

- Conversational, Academic, Storyteller, Motivational, Witty.

2.3 Submit flow

- POST /api/books.
- POST /api/runs/{book_id}/start.
- Redirect to run monitor.

2.4 Validation

- Required fields.
- Chapter count min/max.
- Words per chapter numeric.

Frontend tests

- Cannot submit empty topic.
- Valid form creates book.
- Redirect to run monitor works.
- API errors display nicely.

20. Frontend Module 3 - Run Monitor UI

Show real-time or polling-based status of all agents.

3.1 Agent timeline

- Display Planner, Researcher, Writer, Humanizer, Editor, Fact Checker, Memory Keeper, Assembler, Evals.

3.2 Run status polling

- Poll GET /api/runs/{run_id}.
- Update current agent and progress.

3.3 Error display

- Show failed agent and error message.

3.4 Completion actions

- Enable View Book, View Evals and Export buttons after completed.

Frontend tests

- Timeline displays pending/running/completed.
- Failed run shows error.
- Completed run enables export links.

21. Frontend Module 4 - Book Preview UI

Show generated sections and chapters in readable format.

4.1 Outline viewer

- Show title, subtitle and chapter list.

4.2 Chapter reader

- Display final chapter text.
- Allow switching chapters.

4.3 Section viewer

- Show front matter and back matter sections.

4.4 Insert chapter control

- UI for Test D insert chapter flow.

Frontend tests

- Chapters render in order.
- Front/back matter render.
- Insert chapter form calls API.
- After insertion, chapter order updates.

22. Frontend Module 5 - Memory, Trace, Evals and Export UI

Make backend quality visible to evaluators.

5.1 Memory viewer

- Tabs: Facts, Concepts, Characters, Callbacks, Tone, Decisions.

5.2 Trace viewer

- Table of agent traces with status, time, summaries and errors.

5.3 Prompt log viewer

- Show prompt text safely in expandable panels.

5.4 Evals page

- Show scores for structure, tone, AI tells, facts, callbacks and insertion repair.

5.5 Export center

- Download DOCX, PDF, prompt dossier and trace bundle.

Frontend tests

- Memory tabs render.
- Trace table handles empty state.
- Eval cards show pass/fail.
- Export buttons download files.

23. Final Testing Strategy

Testing Layer	Tool	What to Cover
Backend unit	pytest	Schemas, services, prompt loader, memory services, eval functions.
Backend integration	pytest + test database	API -> DB -> workflow services.
Agent schema tests	pytest with mocked LLM	Each agent returns valid structured output.
RAG tests	pytest	Chunk insertion, embeddings, retrieval, evidence pack generation.
Export tests	pytest	DOCX/PDF files exist and contain required sections.
Frontend unit	React Testing Library	Forms, status components, error rendering.
Frontend E2E	Playwright	Create book, monitor run, view memory/evals, download export.

Acceptance Test Checklist

- Test A generates a 10-chapter beginner personal finance guide in conversational tone.
- Test B generates a 5-chapter novella with two consistent named characters.
- Test C regenerates Chapter 3 in Academic, Motivational and Witty tone while preserving facts.
- Test D inserts a chapter between Chapter 4 and 5 and repairs TOC, callbacks and glossary.
- Every test case has DOCX and PDF output.
- Every test case has trace bundle, prompt log, memory I/O log and token/cost ledger.
- Prompt dossier exists as a standalone output.

24. Copy-Paste Antigravity Master Prompt

```
You are working as a senior full-stack GenAI engineer inside this repository.

Project: AIuthor - Agentic Long-Form Book Generation System.
Stack: React + Vite frontend, FastAPI backend, PostgreSQL + pgvector database, LangGraph orchestration,
Pydantic schemas, python-docx/LibreOffice export, pytest and React/Playwright tests.

Important: Do NOT build this as one giant prompt. Build a production-style agentic workflow with separate
agents: Planner, Researcher, Writer, Humanizer, Editor, Fact Checker, Memory Keeper, and Assembler.

Core requirements:
1. Backend-first implementation.
2. Use PostgreSQL for books, runs, chapters, memory, traces, prompt logs, token/cost ledger and eval
```

```
results.
3. Use pgvector for RAG document chunks and semantic retrieval.
4. Use strict Pydantic schemas for all agent inputs/outputs.
5. Store every prompt in a real prompt file under prompts/.
6. Log every agent call into agent_traces.
7. Log every LLM prompt into prompt_logs.
8. Log every memory read/write into memory_io_logs.
9. Log token usage and estimated cost into token_cost_ledger.
10. Implement chapter insertion repair: inserting a new chapter must repair TOC, callbacks and glossary.
11. Generate DOCX and PDF outputs.
12. Add evals for structure, tone, AI tells, fact grounding, callbacks and insertion repair.


Development rule:
Implement one module at a time. For each module:
- show files created/modified,
- add tests,
- run tests if possible,
- do not remove working code,
- stop after completing the requested module and summarize what was done.


Start with the module I request next. If any required detail is missing, make a reasonable engineering
assumption and document it in docs/decisions.md instead of blocking.
```

25. Module-by-Module Antigravity Prompts

Prompt 1 - Backend Foundation

```
Implement Backend Module 0 and Module 1 only.


Create the repository foundation and FastAPI backend core for AIuthor.


Tasks:
1. Create backend folder structure.
2. Add FastAPI app with /health and /api/version.
3. Add config using environment variables.
4. Add database session dependency.
5. Add CORS for React.
6. Add global error handling.
7. Add .env.example and README setup instructions.
8. Add pytest tests for health endpoint and config loading.


Do not implement agents yet. Do not implement React UI yet. Stop after tests and summary.
```

Prompt 2 - Database and pgvector

```
Implement Backend Module 2 only.


Add PostgreSQL + pgvector database models and Alembic migrations.


Create tables for:
- book_projects
- book_runs
- chapters
- book_sections
- source_documents
- document_chunks with vector(1536)
- fact_registry
- concept_bible
- character_bible
- callback_index
- tone_fingerprints
```

```
- decision_log
- agent_traces
- prompt_logs
- memory_io_logs
- token_cost_ledger
- eval_results
- export_files

Add migration, model tests, and pgvector extension check. Stop after tests and summary.
```

Prompt 3 - Schemas and API Contracts

```
Implement Backend Module 3 only.

Create strict Pydantic schemas for:
- book creation and response
- run status and agent status
- book outline and chapter contract
- evidence pack
- fact check result
- memory updates
- eval results
- export responses

Add validation tests for invalid tone, invalid chapter count, missing required fields, and unsupported statuses. Stop after tests and summary.
```

Prompt 4 - Prompt Registry and Observability

```
Implement Backend Module 4 and Module 5 only.

Create prompt files for all agents and a prompt registry loader.
Then implement observability services:
- agent trace logging
- prompt logging
- memory I/O logging
- token/cost ledger

Add tests that verify prompt files exist, prompt loader works, Humanizer prompt contains banned AI phrases, Fact Checker prompt contains citation-or-soften rule, and all logs can be written. Stop after tests and summary.
```

Prompt 5 - RAG Pipeline

```
Implement Backend Module 6 only.

Build RAG with PostgreSQL + pgvector:
1. Source document ingestion service.
2. Text chunker.
3. Embedding service interface.
4. document_chunks storage with embedding.
5. Semantic retrieval with top_k.
6. Keyword fallback.
7. EvidencePack builder.
8. Retrieval logs.

Use mock embeddings for tests if API key is unavailable. Add tests for chunking, inserting chunks, similarity retrieval, and missing evidence behavior. Stop after tests and summary.
```

Prompt 6 - Memory System

```
Implement Backend Module 7 only.

Build persistent memory services:
```

```
- Fact registry service
- Concept bible service
- Character bible service
- Callback index service
- Tone fingerprint service
- Decision log service
- Memory reader for relevant chapter context

Every memory read/write must write memory_io_logs. Add tests for writing and retrieving each memory type.
Stop after tests and summary.
```

Prompt 7 - LangGraph Workflow Skeleton

```
Implement Backend Module 8 only.

Create LangGraph workflow skeleton with mocked agent nodes first.
Nodes:
- planner
- researcher
- writer
- humanizer
- editor
- fact_checker
- memory_keeper
- assembler
- evals

Add BookState. Add conditional edge: if fact_checker returns needs_repair, route to repair path. Update
book_runs current_agent and status. Add tests for happy path, fact-check repair path, and failure path.
Stop after tests and summary.
```

Prompt 8 - Real Agent Implementations

```
Implement Backend Module 9 only.

Replace mocked nodes with real agent service classes using prompt registry and structured outputs.
Implement:
- Planner Agent
- Researcher Agent
- Writer Agent
- Humanizer Agent
- Editor Agent
- Fact Checker Agent
- Memory Keeper Agent
- Assembler Agent

Each agent must log trace, prompt, token/cost and return strict schema. Use safe mocked LLM mode for tests.
Add tests for each agent schema and key behavior. Stop after tests and summary.
```

Prompt 9 - Chapter Insertion Repair

```
Implement Backend Module 10 only.

Build Test D repair flow:
1. Add POST /api/books/{book_id}/chapters/insert.
2. Insert new chapter after selected chapter.
3. Renumber later chapters.
4. Generate new chapter through the agent workflow.
5. Repair callback_index.
6. Rebuild glossary from concept_bible.
7. Rebuild TOC/book sections.
8. Re-export DOCX/PDF trigger.
9. Add insertion eval.
```

```
Add tests proving new chapter inserted after Chapter 4, old Chapter 5 becomes Chapter 6, callbacks and
glossary update. Stop after tests and summary.
```

Prompt 10 - Export and Evals

```
Implement Backend Module 11 and Module 12 only.


Build DOCX/PDF export and evals:
- python-docx exporter
- PDF conversion using LibreOffice
- front matter, chapters, back matter assembly
- glossary from concept bible
- references from fact registry
- structure eval
- tone eval
- AI tell eval
- fact eval
- callback eval
- insertion eval


Add tests that DOCX/PDF are generated and evals save results. Stop after tests and summary.
```

Prompt 11 - Test Case Runners

```
Implement Backend Module 13 only.


Create scripts/API helpers for mandatory assessment test cases:
A. 10-chapter beginner personal finance guide, conversational tone.
B. 5-chapter novella, storyteller tone, two named characters.
C. Regenerate Test A Chapter 3 in Academic, Motivational and Witty.
D. Insert a new chapter between Test A Chapter 4 and 5 and repair TOC/callbacks/glossary.


Each runner must export DOCX, PDF, eval report and trace bundle. Stop after tests and summary.
```

Prompt 12 - React UI

```
Implement the React UI after backend modules are ready.


Pages:
- Dashboard
- New Book Brief
- Run Monitor
- Book Preview
- Memory Viewer
- Trace Viewer
- Evals Report
- Export Center


Use React + Vite + Tailwind. Keep UI clean and practical. Connect to existing FastAPI endpoints. Add form
validation, loading states and error states. Add component tests and one Playwright E2E flow. Stop after
tests and summary.
```

26. Final Submission Checklist

Deliverable	Status Check
Working MVP	Runs with one command or clear Docker commands.
Source repo	Honest commit history and clear README.
Prompt dossier	Every agent prompt included with purpose, inputs, outputs and failure modes.
Architecture doc	One diagram and one-page data flow/failure path explanation.

Memory schema	Concrete tables and example records.
Evals report	Rubric, scores and failure analysis.
Trace bundle	Agent traces, prompt logs, memory I/O and token/cost ledger.
Sample books	PDF and DOCX for Test A, B, C and D.
Demo video	5-8 minutes showing state, logs and output.
Design decisions log	Ten major implementation decisions and why.

Recommended final message in README: This project demonstrates a backend-first agentic book generation system using real orchestration, structured memory, RAG grounding, humanization, fact checking, self-healing chapter insertion, DOCX/PDF export and full observability.